Rapid k-NN

Ninad Parikh[1], Yash Amin[2], Mohammed Amin Kalota[3], Anjali Jivani[4]

[1] Department of Computer Science and Engineering, The Maharaja Sayajirao University of Baroda. Vadodara, India. Email – parikhninad@gmail.com
[2] Department of Computer Science and Engineering, The Maharaja Sayajirao University of Baroda. Vadodara, India. Email - yashamin88@gmail.com
[3] Department of Computer Science and Engineering, The Maharaja Sayajirao University of Baroda. Vadodara, India. Email – mohammedamink4@gmail.com
[4] Department of Computer Science and Engineering, The Maharaja Sayajirao University of Baroda. Vadodara, India. Email – anjali.jivani-cse@msubaroda.ac.in[4]

ABSTRACT: There are numerous classification algorithms in Machine Learning, each with its own strengths and limitations. Due to this, several optimized variants of these algorithms have been developed to improve efficiency and scalability. This paper introduces and studies the performance of Rapid k-Nearest Neighbours an enhanced variant of the traditional k-Nearest Neighbours (k-NN) algorithm. Rapid KNN utilizes dynamic sampling to reduce dataset size while maintaining class distribution balance, along with approximate nearest Neighbour indexing using PyNNDescent [21] for faster query processing.

The performance of Rapid KNN is evaluated on the Loan Defaulter dataset, which is a very large dataset sourced from Kaggle, which consists of 307,511 rows and 122 columns. We compare Rapid KNN with traditional k-NN and other classification algorithms such as Naïve Bayes, Support Vector Machine (SVM), and Decision Tree Classifier.

Keywords: Machine Learning, Supervised Learning, k-Nearest Neighbours, Rapid k-NN, Naïve Bayes, SVM, Decision Tree, Dynamic Sampling, Approximate Nearest Neighbours, PyNNDescent [21].

1. INTRODUCTION

Machine Learning models play a crucial role in analysing and making predictions from large-scale datasets. These models are broadly categorized into supervised, unsupervised, and reinforcement learning techniques. Among them, supervised learning is one of the most widely used approaches, where the model is trained on labelled data to learn patterns and make predictions. Common supervised learning algorithms include Decision Trees [22], Support Vector Machines (SVM) [23], Naïve Bayes [24], and k-Nearest Neighbours (k-NN) [25].

k-NN is a simple yet effective non-parametric algorithm used for classification and regression tasks. It operates by finding the k closest data points (neighbours) to a given input and predicting the output based on their labels. However, despite its effectiveness, traditional k-NN suffers from severe computational inefficiencies when applied to large datasets. Since k-NN requires computing distances between all data points for every new query, it becomes highly memory-intensive and slow, limiting its scalability in real-world applications.

To overcome these challenges, we propose Rapid k-Nearest Neighbours an optimized variant that improves efficiency while preserving predictive performance. Rapid KNN dynamically samples a subset of the training data, ensuring a balanced representation of different classes, and utilizes approximate nearest Neighbour (ANN) indexing via PyNNDescent [21] to accelerate Neighbour search. By reducing the dataset size and employing efficient indexing, Rapid KNN significantly lowers computational cost without compromising accuracy.

In this study, we evaluate the performance of Rapid KNN on the Loan Defaulter dataset [26], which is a large dataset consisting of 307,511 rows. The results demonstrate that Rapid KNN maintains classification accuracy while significantly improving processing speed and scalability, making it a practical alternative for large-scale machine learning applications.

2. BACKGROUND & RELATED WORK

2.1 k-Nearest Neighbours (k-NN) and Its Applications:

k-Nearest Neighbours (k-NN) is a non-parametric, instance-based learning algorithm used in both classification and regression tasks.

The algorithm classifies new data points by measuring the distance (Euclidean, Manhattan, etc.) between the input and all training samples, selecting the k closest Neighbours, and making predictions based on their majority label (classification) or average value (regression).

2.2 Challenges in Using k-NN for Large Datasets

Despite its simplicity and effectiveness, k-NN becomes inefficient as dataset size increases, due to:

High Computational Complexity – k-NN requires computing the distance of a test sample against all training samples, leading to $O(n \times d)$ complexity, where n is the number of training points and d is the feature dimension.

Memory Usage – Since k-NN is a lazy learner, it stores all training data, consuming significant memory.

Slow Query Time – Searching for Neighbours in high-dimensional data makes inference slower, especially for real-time applications.

2.3 Existing Solutions: Approximate & Optimized k-NN Approaches

Several methods have been proposed to improve k-NN's scalability:

Dimensionality Reduction (e.g., PCA, LSH) – Reduces the feature space to speed up distance calculations.

KD-Trees & Ball Trees – Efficient data structures that partition space to reduce search time but struggle in high dimensions.

Approximate Nearest Neighbour (ANN) Search – Methods like Hierarchical Navigable Small World (HNSW), Annoy, and PyNNDescent [21] build fast, approximate indices for Neighbour retrieval.

Sampling Techniques – Randomly selecting a subset of the training data for distance computation.

2.4 How Rapid KNN Differs from Prior Methods

Rapid KNN combines dynamic sampling with ANN indexing, making it unique from previous k-NN optimizations:

Dynamic Class-Based Sampling – Instead of using the entire dataset, Rapid KNN selects a representative subset based on class distribution. This ensures that minority classes are not underrepresented while reducing computation.

PyNNDescent for Approximate Neighbour Search – After sampling, the algorithm builds an ANN index to quickly retrieve nearest Neighbours, significantly reducing query time.

Scalability & Efficiency – Unlike traditional k-NN, which requires searching the full dataset, Rapid KNN reduces dataset size before indexing, making it more efficient for large-scale data analysis.

3. LITERATURE REVIEW

Cover & Hart (1967) introduced the foundational concept of the k-Nearest Neighbours (k-NN) algorithm, establishing its effectiveness for pattern classification. They demonstrated that k-NN achieves asymptotically optimal classification under certain conditions [1].

Andoni & Indyk (2006) developed near-optimal hashing techniques for approximate nearest neighbour (ANN) search, significantly reducing the computational complexity in high-dimensional spaces [2].

Muja & Lowe (2014) proposed scalable nearest neighbour search algorithms, focusing on optimizing k-NN for large-scale, high-dimensional datasets. Their work introduced fast approximations to reduce computational overhead [3].

Jegou et al. (2010) introduced product quantization for nearest neighbour search, a method that allows high-dimensional data compression while maintaining accurate distance approximations [4].

Jivani (2013) proposed a modified k-NN algorithm to improve efficiency, introducing novel optimizations to reduce search time and memory usage [5].

Dong et al. (2011) presented an efficient method for constructing k-NN graphs, focusing on generic similarity measures to enhance nearest neighbour retrieval speed [6].

Li et al. (2011) explored hashing techniques for large-scale machine learning, providing alternative methods for ANN search through randomized hash functions [7].

Johnson et al. (2019) discussed billion-scale similarity search using GPUs, demonstrating how hardware acceleration significantly speeds up large-scale nearest neighbor computations [8].

McInnes et al. (2017) introduced HDBSCAN, a hierarchical clustering algorithm that extends DBSCAN by providing automatic cluster selection and better handling of noise in data [9].

McInnes et al. (2018) developed UMAP (Uniform Manifold Approximation and Projection), a dimensionality reduction technique that preserves local and global structures more effectively than PCA or t-SNE [10].

Bentley (1975) introduced k-d trees, a fundamental data structure for multidimensional search problems, including efficient nearest neighbour queries [11].

Dasgupta & Freund (2008) proposed random projection trees, a method for handling low-dimensional manifold structures within high-dimensional data [12].

Arya et al. (1998) developed an optimal algorithm for approximate nearest neighbour search, optimizing search efficiency for fixed-dimensional spaces [13].

Biau & Devroye (2015) provided an extensive overview of nearest neighbour methods, covering theoretical properties, applications, and optimizations [14].

Chandola et al. (2009) conducted a survey on anomaly detection techniques, exploring how nearest neighbour methods contribute to identifying outliers in large datasets [15].

Ke et al. (2017) introduced LightGBM, an efficient gradient boosting decision tree (GBDT) framework optimized for large-scale datasets with fast training times [16].

Liu et al. (2008) developed Isolation Forest, a novel anomaly detection algorithm that isolates rare instances instead of modeling normal data distribution [17].

LeCun, Bengio, & Hinton (2015) provided a comprehensive review of deep learning, discussing advancements in neural networks and their impact on AI research [18].

Chawla et al. (2002) introduced SMOTE (Synthetic Minority Over-sampling Technique), a widely used method to handle class imbalance in datasets by generating synthetic samples [19].

McInnes et al. (2017) presented HDBSCAN, a density-based clustering algorithm that improves upon DBSCAN by dynamically determining cluster structures [20].

4. DATA PREPROCESSING

4.1 Data Description

Loan Defaulter dataset [26] consists of 3,07,511 rows and 122 columns. The source of the data is from the Kaggle Website. The target column was identified as TARGET which has the following value counts as 0 has 282,686 and 1 has 24,825. By this dataset we perform risk analytics in banking and financial services and understand how data is used to minimize the risk of losing money while lending to customers. Target variable (1 - client with payment difficulties: he/she had late payment more than X days on at least one of the first Y instalments of the loan, 0 - all other cases)

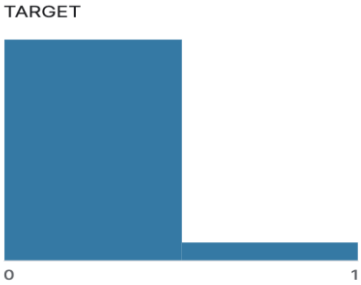

Fig. 1. Loan Defaulter dataset visualized

4.2 Data Splitting

Typically, when splitting a dataset into testing and training sets, the goal is to ensure that no data is shared between the two. This is because the test set's purpose is to simulate real-world, unseen data. However, when evaluating a model, there is full access to both the train and test sets, so it is up to us to ensure that no data in the training set is present in the test set.

4.3 Data Cleaning

Missing Data Threshold: The threshold value for missing data was set to 40%, that means any feature with more than 40% of missing values was removed from our dataset. The reason being, in this case if it was decided to impute such columns, the variance would have been not that great as majority of the data would have been imputed. This reduced the number of features from 122 to 73.

Uncorrelated columns removed: For considerable relation between two features the correlation required is -0.8< to >+0.8. So, columns not having considerable relation with TARGET column will be removed. This reduced the number of features from 73 to 16.

4.4 Feature Engineering

Imputation: Although, the features which had more than 40% of the missing values were eliminated from the dataset, it was required to ensure that the remaining columns must not have missing values as few machine learning models do not accept null values. The missing values are filled by mode for categorical columns, by means/median for numerical columns depending upon the data spread of that respective column.

Transformation: Ordinal Encoding of the categorical columns (ordinal variable) and

scaling down(normalization) the numerical columns are necessary for proper results. The categorical column's order is not specified in the data as such so by proper univariate analysis on those columns we extract the ranks. Then by using Column Transformer we perform both transformations.

5. Rapid KNN: METHODOLOGY & IMPLEMENTATION

5.1 Overview

Rapid KNN is designed to improve the efficiency of traditional k-Nearest Neighbours (k-NN) by reducing dataset size via dynamic sampling and accelerating Neighbour search using Approximate Nearest Neighbour (ANN) indexing. Instead of computing distances for the entire dataset, Rapid KNN selects a representative subset of the data and builds an index for faster retrieval.

This approach significantly reduces computational complexity, memory usage, and query time, making k-NN feasible for large datasets while maintaining competitive accuracy.

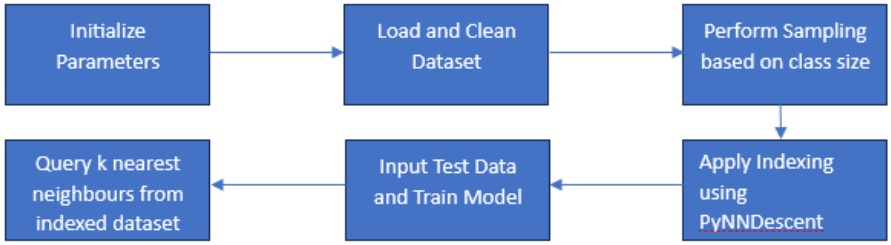

Fig. 2. Working of Rapid KNN

5.2 Rapid KNN Algorithm

Data Sampling (Class-Aware Selection)

To avoid the inefficiencies of using the full dataset, Rapid KNN dynamically selects a subset of training points while ensuring class balance. The sampling process is as follows:

Class Distribution Analysis – Compute the frequency of each class in the dataset.

Dynamic Sampling Ratio – The sampling ratio is adaptive:

- For smaller classes, a higher proportion of data is retained to avoid class imbalance.
- For larger classes, only a fraction of samples is selected to reduce computation.

Random Subset Selection – A stratified random sampling approach ensures that each class is represented adequately in the reduced dataset.

This ensures that minority classes are not underrepresented, preventing bias while reducing overall computation.

Building the Approximate Nearest Neighbour (ANN) Index

After sampling, the algorithm constructs an ANN index using PyNNDescent [21], which efficiently organizes the sampled data for fast Neighbour retrieval. The process involves:

Constructing a Nearest Neighbour Graph – PyNNDescent [21] builds a graph-based structure where nodes represent data points, and edges represent nearest Neighbours.

Efficient Search Operations – Instead of computing distances from all training points, the algorithm navigates the graph to find approximate nearest Neighbours in logarithmic time.

Storage Optimization – Since the full dataset is not stored in memory, memory overhead is significantly reduced compared to traditional k-NN.

Neighbour Selection & Prediction

For a given test point:

Querying the ANN Index – Instead of searching the entire dataset, the ANN index quickly retrieves the k-nearest Neighbours.

Prediction Based on Neighbours –

- For classification, the most frequent class among the Neighbours is selected (mode).
- For regression, the average of the Neighbour values is taken.

This significantly speeds up prediction while ensuring accurate results.

Theoretical Improvements Over Standard k-NN

Rapid KNN introduces two major theoretical improvements:

Reduced Computational Complexity – Since only a subset of data is used, Rapid KNN requires fewer distance computations.

Efficient Query Execution via ANN – Instead of scanning the dataset linearly, the ANN index retrieves Neighbours in sublinear time.

Method	Training Complexity	Query Complexity	Memory Usage
k-NN	O(1) (lazy learning)	O(n × d)	High (stores full dataset)
KD-Tree k-NN	O(n log n)	O(log n) (low dimensions)	Moderate
HNSW k-NN	O(n log n)	O(log n) (high dimensions)	Moderate
Rapid KNN	O(n × r) (sampling) + O(n log n) (indexing)	O(log n)	Low (stores only sampled data)

Fig. 3. Comparison of Rapid KNN with other KNN Algorithms

6. RESULTS

In scenarios where the dataset is relatively small, the advantages of Rapid KNN might not be as pronounced compared to its traditional counterpart. The fundamental strength of Rapid KNN lies in its ability to significantly accelerate the computation process, especially with large datasets where the time complexity of KNN can become a bottleneck.

In a smaller dataset, the time saved by Rapid KNN might not be as apparent since the computational overhead of the traditional KNN in such cases is already manageable. However, it's crucial to recognize that the true potential of Rapid KNN shines in

situations with vast datasets, where its optimized algorithms lead to substantial performance improvements.

Consequently, to comprehensively showcase the prowess of Rapid KNN, we conducted extensive tests on a sizable dataset containing over 300,000 rows. The outcomes of these tests, detailed in the subsequent sections, provide a clear visual representation of the performance benefits reaped by Rapid KNN in scenarios where the dataset scales up, further validating its efficiency and relevance in handling substantial real-world datasets.

Pyndescent for Large Datasets:

For extensive datasets, we leverage the Pyndescent library to enhance the performance of our Rapid KNN implementation. Pyndescent efficiently handles large datasets, providing scalability and improved computation speed.

Pyndescent for Sparse Datasets:

In scenarios where the dataset is sparse, Pyndescent's approach of creating a Numpy array might incur additional processing time. To address this, traditional KNN methods utilizing structures like ball trees are more efficient. Hence, in sparse datasets, we should shift to traditional KNN with boosting techniques for optimized performance

	CV=2	CV=3	CV=3
Traditional KNN	91.77	91.75	91.77
Rapid KNN	91.79	91.80	91.79

Fig. 4. Comparison for different CV Parameter

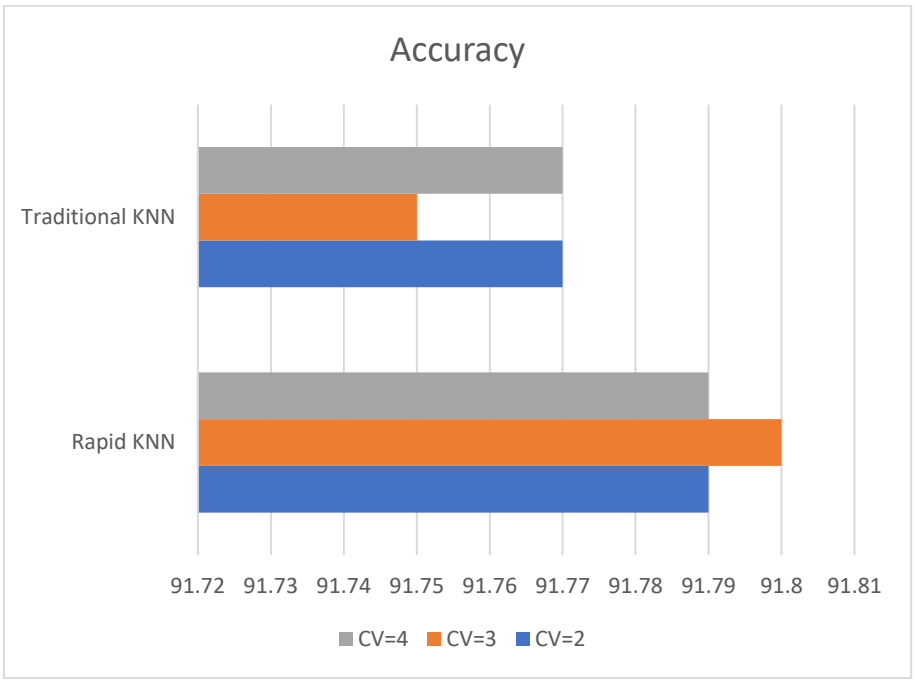

Fig. 5. Comparison of Accuracy for Rapid KNN to Traditional KNN

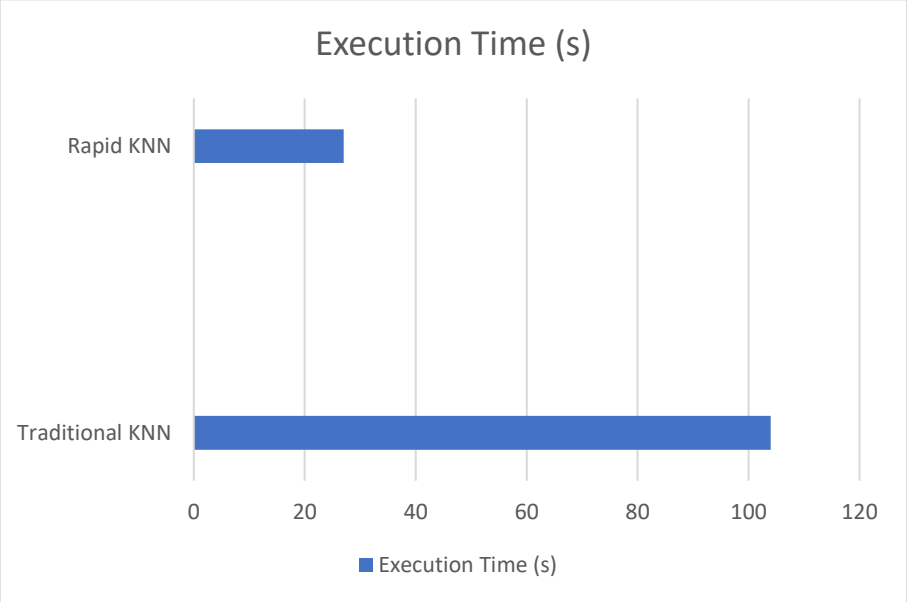

Fig. 6. Comparison of Execution time for Rapid KNN to Traditional KNN

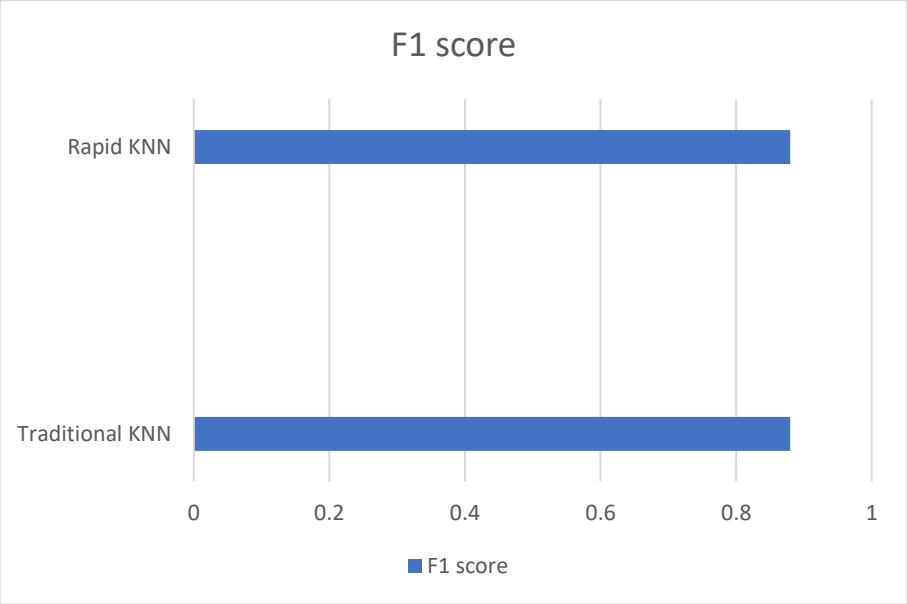

Fig. 7. Comparison of F1 Score for Rapid KNN to Traditional KNN

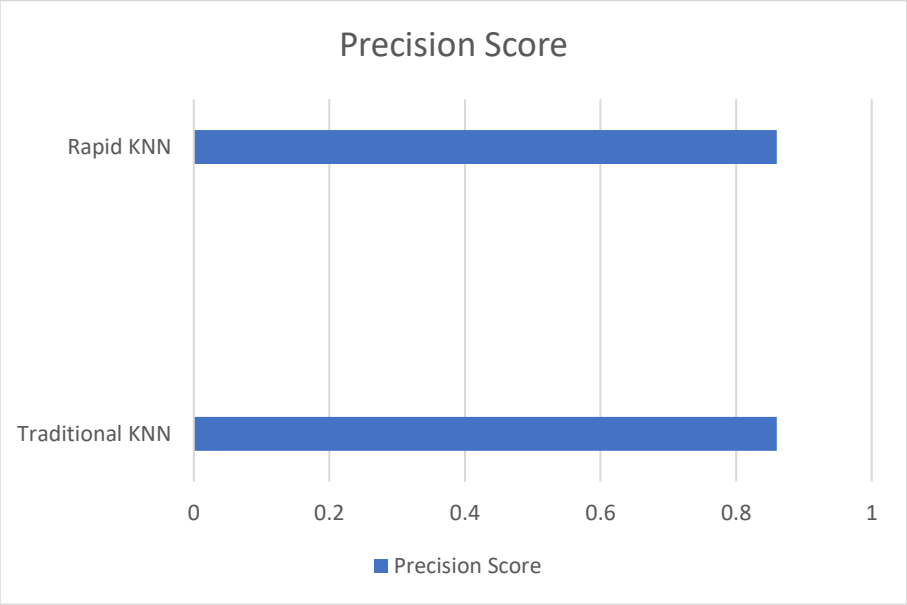

Fig. 8. Comparison of Precision Score for Rapid KNN to Traditional KNN

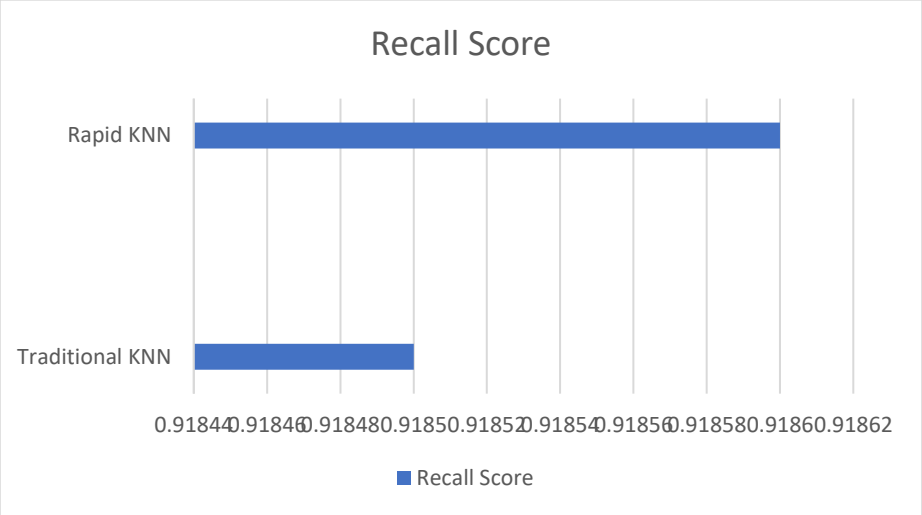

Fig. 9. Comparison of Recall Score for Rapid KNN to Traditional KNN

7. CONCLUSION

In this study, we introduced Rapid k-Nearest Neighbours (Rapid KNN), an optimized variant of the traditional k-NN algorithm designed to improve efficiency in large-scale classification tasks. By incorporating dynamic class-based sampling and Approximate Nearest Neighbour (ANN) indexing using PyNNDescent [21], our approach successfully mitigates the computational and memory constraints associated with k-NN.

Experimental evaluations on the Loan Defaulter dataset [26] demonstrated that Rapid KNN achieves comparable classification accuracy while significantly reducing computational complexity and query time. The key improvements of Rapid KNN over standard k-NN include:

- Reduced computational overhead through adaptive sampling, which selects a representative subset of training data.
- Faster query execution via ANN-based indexing, which enables efficient nearest Neighbour retrieval.
- Improved scalability by minimizing memory usage while preserving predictive performance.

Possible Enhancements:

- Adaptive Sampling Strategy: Instead of a fixed ratio, adjust sampling based on class imbalance severity.
- Weighted Neighbour Contributions: Instead of using mode (majority vote) for classification, assign weights based on distance (closer Neighbours have more influence).

8. REFERENCES

[1] Cover, T., & Hart, P. (1967). *Nearest neighbour pattern classification*. IEEE Transactions on Information Theory, 13(1), 21-27.

[2] Andoni, A., & Indyk, P. (2006). *Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions*. In Proceedings of the 47th

Annual IEEE Symposium on Foundations of Computer Science (FOCS), 459-468.

[3] Muja, M., & Lowe, D. G. (2014). *Scalable nearest neighbor algorithms for high dimensional data*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 36(11), 2227-2240.

[4] Jegou, H., Douze, M., & Schmid, C. (2010). *Product quantization for nearest neighbor search*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1), 117-128.

[5] Jivani, A. (2013). *The Novel k Nearest Neighbor Algorithm*. International Conference on Computer Communication and Informatics (ICCCI).

[6] Dong, W., Wang, Z., Charikar, M., & Li, K. (2011). *Efficient k-nearest neighbor graph construction for generic similarity measures*. In Proceedings of the 20th International Conference on World Wide Web (WWW), 577-586.

[7] Li, P., Shrivastava, A., Moore, J., & Konig, A. C. (2011). *Hashing algorithms for large-scale machine learning*. Advances in Neural Information Processing Systems (NeurIPS), 24.

[8] Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 7(3), 535-547.

[9] McInnes, L., Healy, J., & Astels, S. (2017). *hdbscan: Hierarchical density based clustering*. Journal of Open Source Software, 2(11), 205.

[10] McInnes, L., Healy, J., Saul, N., & Großberger, L. (2018). *UMAP: Uniform manifold approximation and projection*. The Journal of Open Source Software, 3(29), 861.

[11] Bentley, J. L. (1975). *Multidimensional binary search trees used for associative searching*. Communications of the ACM, 18(9), 509-517.

[12] Dasgupta, S., & Freund, Y. (2008). *Random projection trees and low-dimensional manifolds*. In Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC), 537-546.

[13] Arya, S., Mount, D. M., Netanyahu, N. S., Silverman, R., & Wu, A. Y. (1998). *An optimal algorithm for approximate nearest neighbor searching fixed dimensions*. Journal of the ACM (JACM), 45(6), 891-923.

[14] Biau, G., & Devroye, L. (2015). *Lectures on the nearest neighbor method*. Springer.

[15] Chandola, V., Banerjee, A., & Kumar, V. (2009). *Anomaly detection: A survey*. ACM Computing Surveys (CSUR), 41(3), 1-58.

[16] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., & Liu, T. (2017). *LightGBM: A highly efficient gradient boosting decision tree*. Advances in Neural Information Processing Systems (NeurIPS), 30.

[17] F. T. Liu, K. M. Ting and Z. -H. Zhou, *Isolation Forest*, 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413-422.

[18] LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. Nature, 521(7553), 436-444.

[19] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic minority over-sampling technique*. Journal of Artificial Intelligence Research, 16, 321-357.
[20] McInnes, L., Healy, J., & Astels, S. (2017). *HDBSCAN: Hierarchical density based clustering*. Journal of Open Source Software, 2(11), 205.
[21] https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html
[22] https://scikit-learn.org/stable/modules/tree.html
[23] https://scikit-learn.org/stable/modules/svm.html
[24] https://scikit-learn.org/stable/modules/naive_bayes.html
[25] https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html
[26] https://www.kaggle.com/datasets/gauravduttakiit/loan-defaulter?select=application_data.csv